

EduPredict AI

Student Performance Prediction and Early Intervention System

Fundamentals of AI and ML | VITyarthi Build Your Own Project

Academic Project Report

1. Introduction

EduPredict AI is an educational machine learning application designed to demonstrate an end-to-end AI/ML workflow. The system accepts academic and behavioral indicators, validates and processes them, predicts an expected final score, classifies a risk level, and generates structured recommendations. A Streamlit dashboard provides analytics and saved prediction records.

2. Problem Statement

Educational performance is often reviewed after assessments. This project explores a data-driven approach for organizing student indicators and using machine learning to estimate performance and identify potential risk levels early enough for structured academic intervention.

3. Objectives

- Manage student information and prediction records.
- Perform data cleaning and exploratory analysis.
- Train regression and classification models.
- Evaluate models using appropriate metrics.
- Generate predictions and recommendations.
- Present results through a user-friendly dashboard.

4. Functional Requirements

FR1: Add and validate student information.
FR2: Process academic features for ML.
FR3: Predict final score using regression.
FR4: Classify risk level using classification.
FR5: Generate recommendations from student indicators and predicted risk.
FR6: Display analytics and model results.
FR7: Store prediction records in SQLite.

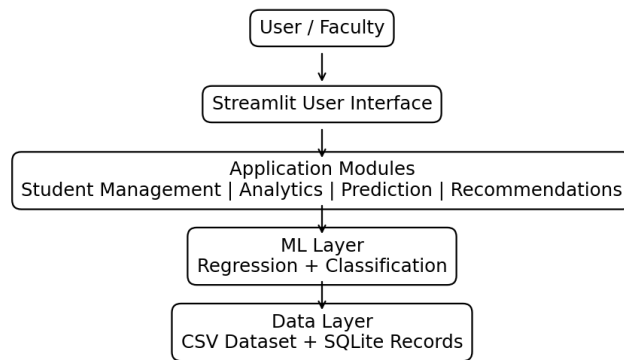
5. Non-Functional Requirements

Performance: predictions should return within normal interactive application response time.
Usability: inputs are grouped into clear dashboard sections.
Reliability: invalid ranges are rejected and errors are displayed.
Maintainability: preprocessing, training, prediction, recommendation, database, and UI logic are separated into modules.
Resource efficiency: lightweight scikit-learn models and CSV/SQLite storage are used.
Error handling: validation prevents impossible values such as attendance above 100%.

6. System Architecture

The application uses a layered structure. Streamlit provides the interface. Application modules handle data management, analytics, prediction and recommendations. Scikit-learn models form the ML layer. CSV data supports the training dataset while SQLite stores generated prediction records.

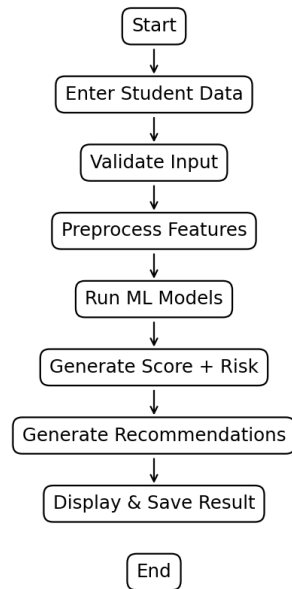
EduPredict AI - System Architecture



7. Design Diagrams

The architecture and workflow diagrams are included in the repository under report/. The project also defines the major components Student/Data Preprocessor/ML Model/Prediction Engine/Recommendation Engine/Database Manager/Analytics.

EduPredict AI - Workflow



8. Dataset Description

The included dataset contains 500 synthetic student records with attendance, study hours, assignment score, internal marks, previous score, backlogs, participation, final score, and risk level. The synthetic dataset makes the repository self-contained for demonstration. A real externally sourced dataset should only be used when approved and documented by the instructor.

9. Model Selection and Methodology

Two regression models are trained: Linear Regression and Random Forest Regressor. Two classification models are trained: Logistic Regression and Random Forest Classifier. Data is split into training and testing subsets with a fixed random seed. Standard scaling is used within linear/logistic pipelines. The training script selects the regression model with the lowest RMSE and the classification model with the highest weighted F1 score.

10. Evaluation

Best regression model selected by the training script: LinearRegression. Best classification model selected by the training script: LogisticRegression. Detailed comparison tables are stored in `models/regression_results.csv` and `models/classification_results.csv`.

11. Implementation Details

The project is implemented in Python. Pandas handles data loading and cleaning; Scikit-learn provides models, preprocessing and evaluation; Streamlit provides the UI; SQLite stores prediction records; Joblib serializes trained models; Pytest provides validation tests.

12. Screenshots / Results

The repository includes EDA plots in `screenshots/`. When the application is run, capture screenshots of the Dashboard, Prediction results, Analytics, and Student Records pages and place them in `screenshots/` before final submission.

13. Testing Approach

Automated tests cover valid input validation, invalid attendance handling, and recommendation generation. Manual testing should additionally cover empty fields, out-of-range values, prediction generation, database storage, and dashboard navigation.

14. Challenges Faced

Key implementation challenges include selecting meaningful features, keeping preprocessing consistent between training and prediction, validating user inputs, separating model logic from UI logic, and ensuring the project remains reproducible with a self-contained dataset.

15. Learnings & Key Takeaways

The project demonstrates the practical ML lifecycle from dataset preparation through model evaluation and deployment in a simple application. It also reinforces modular Python design, validation, model persistence, analytics, testing, and Git-based project organization.

16. Future Enhancements

Future versions could support approved real institutional datasets, subject-wise prediction, automated progress tracking across semesters, explainable-AI feature importance, mentor notifications, authentication, and scheduled retraining.

17. References

Scikit-learn documentation; Pandas documentation; Streamlit documentation; Python documentation. Exact external references should be formatted according to the citation style required by the course/instructor.